Project M1 S2 – 2021

Design, manufacture, and test of a three-dimensional cell culture system

Documentation – BIOCAD Interface

Supervisor : Vincent SENEZ

Rémi **ARBACHE** remi.arbache@student.junia.com
Eugénie **DALMAS** eugenie.dalmas@student.junia.com

TABLE OF CONTENTS

Introduction 2
I. BIOCAD application 3
1. Properties 3
a. UI components 3
b. Variables 7
2. Methods 8
3. Event Callback functions 12
a. Created 12
b. Generated 17
II. Utils functions 18
III. SRC directory 18
IV. Assets 19
V. Tricks of the trade 19
Index – Functions 20

Introduction

Our project is part of pancreatic cancer research. We designed and developed a software interface allowing to import a microfluidic system with electrodes, as well as a set of cells, to run an electrical simulation of cell impedance. We built the interface with Matlab's App Designer and integrated a pre-existing SPICE simulation engine. The interface aims to decrease the simulation time using SPICE simulations that are much faster than finite elements simulations (such as those available in COMSOL). Besides, the interface allows the addition of approximate cells in the form of ellipsoids, derived from confocal microscopy, to the simulated geometry.

This document is the code documentation of the BIOCAD application: all properties, methods, and event callbacks of the BIOCAD app will be identified and explained. Moreover, the other files used by the app will be introduced, such as those in the `/Utils`, `/Assets` or `/src` folders of the `/BIOCAD` package.

The source code can be found on GitHub in the private repository /RemiArbache/BIOCAD. To access it, it is necessary to be a collaborator.

I. BIOCAD application

The application `biocad.mlapp` is the main interface that the user will use, with 3 tabs: Menu (initialize, load, or import parameters) ; Parameters (define some of the parameters for simple shapes and fewer for complex ones) ; Plot (result section as a plot and reminder of chosen parameters if done in the interface).

1. Properties

All the properties are created or auto generated by the App Designer when an integrated UI component is dragged and dropped on the interface (Design View of the App Designer). In this manner, all property, except some Label components, will be described.

a. UI components

They are auto generated components present on the interface, dragged and dropped from Matlab App Designer integrated components. Their access is public thus those components are accessible outside the application code such as matlab scripts.

- **Global components**

`UIFigure` (matlab.ui.Figure)
Main figure parent which all components are linked to.

`FoldernameLabel` (matlab.ui.control.Label)
The folder name of the current chosen working directory.

`CopyrightsLabel` (matlab.ui.control.Label)
Application owner's name and main purpose of the application.

`MainTabGroup` (matlab.ui.container.TabGroup)
Tab group containing the sections Menu, Parameters and Plot.

`ContextMenu` (matlab.ui.container.ContextMenu)
Context menu that opens when right-clicked on UI Objects that are linked to it. More than one menu can be created and linked to different components, but for now only one context menu is defined.

`OpeninexternalfigureMenu` (matlab.ui.container.Menu)
Menu available in its Context Menu parent component and that is linked to UI axis. Open the linked axis in the external figure.

- **Menu section**

`LoadingPanel` (matlab.ui.container.Panel)
Temporary panel that is destroy when the application is completely loaded. Prevent user to click on buttons or tabs.

`LoadingLabel` (matlab.ui.control.Label)

`StoreDirectoryButtonGroup` (matlab.ui.container.ButtonGroup)
Parent of the component for the selection of the store directory for the results files.

`FilePathLabel` (matlab.ui.control.Label)
Path of the default or selected store directory.

`BrowseButton` (matlab.ui.control.Button)
Button that opens a browsing window and where the user selected his store directory.

`AboutButton` (matlab.ui.control.Button)

Button for the development credits. When clicked open a modal notification window with developer's information and gives access to the GitHub repository through a button.

BiocadLogo (matlab.ui.control.Image)
Logo image from /Assets.

LOADLabel (matlab.ui.control.Label)

LoadIMSFolderButton_Menu (matlab.ui.control.Button)
Button to load an IMS folder to import the identified cells' geometry.

LoadMatlabFileButton_Menu (matlab.ui.control.Button)
Button to load a Matlab file (.mat) where the interface parameters are defined. Re-import the IMS and/or COMSOL files if they still have the same names and path.

LoadMPHFileButton_Menu (matlab.ui.control.Button)
Button to load a COMSOL file (.mph) of the system geometry (bioreactor, electrodes and may also contain cells).

ToolsPanel_Menu (matlab.ui.container.Panel)
Panel that does not contains any components yet.

- **Parameters section**

ParametersTab (matlab.ui.container.Panel)
Tab from the MainTabGroup containing all parameters components used for the simulation.

MicroSystemPanel (matlab.ui.container.Panel)
Panel containing some information about the system loaded from mph file as well as a preview.

ElectrodeRolePanel (matlab.ui.container.Panel)
Panel where are contained the components concerning the electrodes' role.

CurrentNegativeDropDown_E (matlab.ui.control.DropDown)
Dropdown of the available electrodes' id for the Current - role.

CurrentNegativeDropDownLabel_E (matlab.ui.control.Label)

CurrentPositiveDropDown_E (matlab.ui.control.DropDown)
Dropdown of the available electrodes' id for the Current + role.

CurrentPositiveDropDownLabel_E (matlab.ui.control.Label)

VDDropDown_E (matlab.ui.control.Dropdown)
Dropdown of the available electrodes' id for the VDD role.

VDDDropDownLabel_E (matlab.ui.control.Label)

V0DropDown_E (matlab.ui.control.DropDown)
Dropdown of the available electrodes' id for the V0 role.

V0DropDownLabel_E (matlab.ui.control.Label)

FileInfoLabel_MicroSystem (matlab.ui.control.Label)
Title label for the written information.

FileInfoEntryLabel_MicroSystem (matlab.ui.control.Label)
Name of the loaded mph file.

ClearButton_MicroSystem (matlab.ui.control.Button)
Clear button for the COMSOL model of the microsystem imported.

ImportButton_MicroSystem (matlab.ui.control.Button)
Button to import a COMSOL file for the microsystem.

UIAxes_MicroSystem (matlab.ui.control.UIAxis)
Axes for the preview of the system geometry.

BiocellsPanel (matlab.ui.container.Panel)

Panel containing some information of the cells loaded from a IMS folder as well as a preview.

DistanceToReferentialLabel (matlab.ui.control.Label)

DistanceToReferentialEditField_Z (matlab.ui.control.NumericEditField)

Editable numeric entry for the distance X to the referential point that should be in the COMSOL geometry model of the microsystem.

DistanceToReferentialEditField_Y (matlab.ui.control.NumericEditField)

Editable numeric entry for the distance Y to the referential point.

DistanceToReferentialEditField_X (matlab.ui.control.NumericEditField)

Editable numeric entry for the distance X to the referential point.

SaveCopyCheckBox (matlab.ui.control.Checkbox)

Checkbox that generates a copy of the loaded mph file (microsystem) when checked for the adding of the IMS cells in the microsystem model.

AddcellstomodelButton (matlab.ui.control.Button)

Button that adds the current loaded cells approximation to the current COMSOL model of the microsystem with the given distance to the Imaris referential point.

FileInfoLabel_Biocells (matlab.ui.control.Label)

Title label for the written information.

FileInfoEntryLabel_Biocells (matlab.ui.control.Label)

Name of the loaded Imaris file and the number of approximated cells.

ClearButton_Biocells (matlab.ui.control.Button)

Clear button of the loaded IMS cells.

ImportButton_Biocells (matlab.ui.control.Button)

Import button for the Imaris folder containing the csv file of the approximated IMS cells.

UIAxes_Biocells (matlab.ui.control.UIAxis)

Axes for the preview of the cells' geometry.

ElectricalComponentsPanel (matlab.ui.container.Panel)

Panel containing physics parameters (electric properties) of the global system.

PointsperdecadeEditField (matlab.ui.control.NumericEditField)

Field for the points per decade entry.

PointsperdecadeEditFieldLabel (matlab.ui.control.Label)

VoltageUnitDropDown (matlab.ui.control.DropDown)

Drop down for the unit selection of the offset voltage (V ; mV ; µV ; nV).

VoltageEditField (matlab.ui.control.NumericEditField)

Field for the offset voltage entry.

VoltageEditFieldLabel (matlab.ui.control.Label)

maxFrequencyUnitDropDown (matlab.ui.control.DropDown)

Drop down for the unit selection of the max frequency (Hz ; kHz ; MHz).

maxFrequencyEditField (matlab.ui.control.NumericEditField)

Field for the max frequency entry.

maxFrequencyEditFieldLabel (matlab.ui.control.Label)

minFrequencyUnitDropDown (matlab.ui.control.DropDown)

Drop down for the unit selection of the max frequency (Hz ; kHz ; MHz).

minFrequencyEditField (matlab.ui.control.NumericEditField)

Field for the min frequency entry.

minFrequencyEditFieldLabel (matlab.ui.control.Label)

PHYSICSLabel (matlab.ui.control.Label)

GEOMETRYLabel (matlab.ui.control.Label)

ToolsPanel_Parameters (matlab.ui.container.Panel)
Panel containing tools components within or not section panels.
LogCheckBox (matlab.ui.control.CheckBox)
Checkbox to check if the user wants to generate a log file from the executed simulation.
LoadMPHFileButton_Parameters (matlab.ui.control.Button)
Import button for the COMSOL file of the microsystem.
SaveButton (matlab.ui.control.Button)
Save button for the current entered parameters (edit fields, selections, path of the imported files, corking directory)
ImportfromLabel (matlab.ui.control.Label)
Label specifying the use of the buttons below it.
PreviewallButton (matlab.ui.control.Button)
Button that opens a figure and plot all the geometry that can be plotted in Matlab (cells from Imaris, geometry created from the interface).
RUNLabel (matlab.ui.control.Label)
RunButton (matlab.ui.control.Button)
Button to run the simulation (*compute* function file).
SystemPanel (matlab.ui.container.Panel)
Panel containing buttons related to the system (run and preview).
SYSTEMLabel (matlab.ui.control.Label)
EntryProbeLabel (matlab.ui.control.Label)
Label that specifies the chosen number of electrodes.
ProbeLabel (matlab.ui.control.Label)
EntryDimensionLabel (matlab.ui.control.Label)
Label that specifies the chosen dimension (3D or 2D).
DimensionLabel (matlab.ui.control.Label)
MESHLabel (matlab.ui.control.Label)
MeshdensitySlider (matlab.ui.control.Slider)
Slider that makes the user select an integer number for the mesh density between 1 and 9.
MeshdensitySliderLabel (matlab.ui.control.Label)
StudiesButtonGroup (matlab.ui.container.ButtonGroup)
Group of buttons with multiple selection to choose the studies that will be ran in computation.
TransportLatticeListBox (matlab.ui.control.ListBox)
List box containing a single selected disabled item, Transport Lattice, since the SPICE simulation is always run.
STUDIESLabel (matlab.ui.control.Label)
StudiesListBox (matlab.ui.control.ListBox)
Choices of studies (Analytical or Finite elements).
FILELabel (matlab.ui.control.Label)
LoadIMSFolderButton_Parameters (matlab.ui.control.Button)
Button to load the cells from a Imaris generated folder.
LoadMatlabFileButton_Parameters (matlab.ui.control.Button)
Button to load all parameters saved as a .mat file from the interface.

- **<u>Plot section</u>**

SpicePlotsTab (matlab.ui.container.Tab)
Tab for the results generated by the SPICE simulation (Transport Lattice study).
TabGroup (matlab.ui.container.TabGroup)

Group of the following tabs for all results axes generated by the SPICE simulation (Model, Bode, Nyquist, Potential Map f_{min}, Potential Map fmax).

`ModelTab` (matlab.ui.container.Tab)

Tab contains the geometry and mesh plots of the COMSOL model used for the simulation (with or without IMS cells).

`UIAxesMesh_ModelResult` (matlab.ui.control.UIAxes)

Axes of the model's mesh.

`UIAxesGeometry_ModelResult` (matlab.ui.control.UIAxes)

Axes of the model's geometry.

`BodeTab` (matlab.ui.container.Tab)

Tab containing the bode diagrams in magnitude and phase.

`SPICEBodeDiragramforElectricalImpedanceLabel` (matlab.ui.control.Label)

Title of the tab.

`UIAxesPhase_Bode` (matlab.ui.control.UIAxes)

Axes of the Bode diagram in phase.

`UIAxesMagnitude_Bode` (matlab.ui.control.UIAxes)

Axes of the Bode diagram in magnitude.

`NyquistTab` (matlab.ui.container.Tab)

Tab containing the Nyquist diagram.

`UIAxes_Nyquist` (matlab.ui.control.UIAxes)

Axes of the Nyquist diagram.

`PotMapfminTab` (matlab.ui.container.Tab)

Tab containing the Potential Map for f = f_{min}.

`UIAxes_PotentialMapfmin` (matlab.ui.control.UIAxes)

Axes of the Potential Map for f = f_{min}.

`PotMapfmaxTab` (matlab.ui.container.Tab)

Tab containing the Potential Map for f = f_{max}.

`UIAxes_PotentialMapfmax` (matlab.ui.control.UIAxes)

Axes of the Potential Map for f = f_{max}.

`ToolsPanel_Plot` (matlab.ui.container.Panel)

Panel that contains tools buttons concerning the SPICE plots.

`FILELabel_Plot` (matlab.ui.control.Label)

`SaveAllFiguresButton` (matlab.ui.control.Button)

Button that saves all generated plots visible in the interface (Spice Plot tab) as fig files.

b. Variables

Those variables are the properties that has not been generated by the App Designer. Their access is public just like the UI components, thus can be accessed outside the code of the application.

`settings` (Struct)

Settings parameters: spatial dimension (`settings.dimensions`), number of electrodes (`settings.probes`), working directory path (`settings.workingDir`), source code directory path (`settings.srcDir`).

`externalFigure` (Figure)

Figure that is used to plot axis in another figure window. Created to keep the number of possible windows opened simultaneously to 2 (app window and this figure window).

`cells` (Struct)

Variables that hold the information retrieved from csv files of a loaded IMS folder about the approximated cells by Imaris. The structure is composed of:

`cells.number` : number of cells approximated in the loaded IMS folder or in the system.

`cells.angles_axisA` : lists of rotations of the direction A in the (x,y,z) coordinates system for each cell.

`cells.size` : lists `cells.size.X`, `cells.size.Y` and `cells.size.Z` of radii in each local direction of the approximated cells (ellipsoids).

`cells.direction` : lists of vector coordinates on the local axis A, B and C of the approximated cells (ellipsoids).
ex : `cells.direction.A.X, cells.direction.A.Y and cells.direction.A.Z`

`cells.pos` : lists X, Y and Z of the positions of the approximated cells (ellipsoids).

`cells.distance_to_scan_referential` : list [dx dy dz] of the distances in the (x,y,z) coordinates system to a fixed referential point in the COMSOL model.

`cells.filename` : name of the loaded Imaris file.

`loadedModel` (Struct)

Information about the loaded COMSOL model for the microsystem: COMSOL model (`loadedModel.model`), filename (`loadedModel.filename`), and path (`loadedModel.path`).

`lastAxisClicked` (UIAxes)

Axes Object that has been lastly right clicked on.

`lastResultFoldername` (String)

Name of the last result folder generated by the simulation and containing the spi, spi_cou or fig files.

2. Methods

The created methods are public for simplicity. Moreover, the app is always an input parameter of all methods as it is mandatory. It specifies the instance of the biocad app (self app) as a reference.

initCells

Initializes the cells property as a struct.

S Y N T A X
```
initCells(app)
```

D E S C R I P T I O N

- Resets the fields of the `cells` struct to default values.

getChildren

Recursively gets all ui children of a given parent that has the given ui type.

S Y N T A X
```
[components] = getChildren(app, parent, type)
```

EXAMPLE

```
uiAxes = getChildren(app, app.SpicePlotsTab, "axes");
```

Returns all objects of type `"axes"` that are children of the Spice Plots Tab. Returns axes in the uiAxes variable.

refreshParametersDisplay

Changes the available parameters given the system parameters at creation/load of parameters.

SYNTAX

```
refreshParametersDisplay(app)
```

DESCRIPTION

- Refreshes the dimension, working directory, and number of probes display, aswell as the electrodes' settings by fetching the `settings` variable.

SEE ALSO

`settings`

dispFileInfo

Updates the file information depending on the panel.

SYNTAX

```
dispFileInfo(app, panelName)
```

- With `panelName` = `"biocells"` or "system".

DESCRIPTION

- Displays relevant file information in the given panel.

SEE ALSO

`cells`, `loadedModel`

appendChildrenToArray

Appends all editable children's values from given parent to given array.

SYNTAX

```
[array] = appendChildrenToArray(app, parent, array)
```

DESCRIPTION

- Gets all elements of type `"dropdown"` or `"numericeditfield"` and appends their value to the given array.
- Returns filled array.

setValuesFromArray

Sets all editable fields of given parent to values from given array.
This function relies on the order in which values were saved: saved files will not be compatible after an interface update.

S Y N T A X
```
setValuesFromArray(app, parent, array)
```

D E S C R I P T I O N
- Assigns array values to parent's editable children in order.

saveParameters

Saves all parameters to user-specified file.

S Y N T A X
```
saveParameters(app)
```

D E S C R I P T I O N
- Saves all relevant information submitted to the interface to a *.mat* file.
  This includes : the `settings` and `cells` variables, and the values of the different fields. A COMSOL object cannot be saved in a *.mat* file, so only the path to the model is saved.

S E E A L S O
appendChildrenToArray, settings, loadedModel, cells, loadParameters

loadParameters

Fills interface with parameters from .mat file.

S Y N T A X
```
[success] = loadParameters(app)
```

D E S C R I P T I O N
- Lets the user choose a *.mat* file previously saved from the app using saveParameters, loads the stored data and refreshes what is displayed.

S E E A L S O
saveParameters, settings, loadedModel, cells

countElements

Counts the number of elements from the loaded COMSOL model that have a label containing the given string.

S Y N T A X
```
[count] = countElements(app, type)
```
- With type = `"Cell" or "Electrode" or "Bioreactor" or "Pt"`

D E S C R I P T I O N

- Parses the geometry of the loaded COMSOL object for individual component labels and counts those corresponding to the input.
- Returns the count.

S E E A L S O
loadedModel

initFigure

Initializes the externalFigure from app properties.

S Y N T A X
```
initFigure(app)
```

D E S C R I P T I O N
- Creates an external figure if necessary, otherwise just shows the existing one.
- Used to limit the number of external figures to 1.

S E E A L S O
externalFigure

IMSCellPreview

Plots the geometry of each loaded cell from IMS folder as an ellipsoid.

S Y N T A X
```
IMSCellPreview(app, ax)
```

D E S C R I P T I O N
- Displays a 3D preview of the imported cells in the given axes.

S E E A L S O
cells

checkIMSFiles

Checks if all mandatory and needed csv files are in the selected IMS folder.

S Y N T A X
```
[success] = checkIMSFiles(app, dirpath)
```

D E S C R I P T I O N
- Checks if the following files exist in the given directory :
  - *<name>*_Position.csv
  - *<name>*_Ellipsoid_Axis_A.csv
  - *<name>*_Ellipsoid_Axis_B.csv
  - *<name>*_Ellipsoid_Axis_C.csv
  - *<name>*_Ellipsoid_Axis_Length_A.csv
  - *<name>*_Ellipsoid_Axis_Length_B.csv
  - *<name>*_Ellipsoid_Axis_Length_C.csv

- Returns 1 if all are found, otherwise shows an error pop-up with the name of one missing file.

SEE ALSO
loadIMSFolderButtonPushed, cells

checkMandatoryFields

Checks if all mandatory variables are not null. Used before running.

SYNTAX
```
[success] = checkMandatoryFields(app)
```

DESCRIPTION
- Checks if a COMSOL Model was loaded.
- Checks for 0 values in the Electrical Components Panel.

SEE ALSO
RunButtonPushed, loadedModel

checkFrequencyRange

Checks that the min frequency is lower than the max frequency.

SYNTAX
```
[success] =  checkFrequencyRange(app)
```

SEE ALSO
RunButtonPushed

checkModelCells

Checks if the loaded model already contains IMARIS cells or if cells has been created (in COMSOL) and prevents superposing IMARIS cells.

SYNTAX
```
checkModelCells(app)
```

SEE ALSO
AddcellstomodelButtonPushed, cells, loadedModel

3. Event Callback functions

Those callbacks are generated as private methods. Event only happening with app UI components thus restricted access to the app.

The following callbacks only take as input parameters the app (app : self app) and the triggered event (event : event that has been triggered and has called the callback function).

a. Created

For each UI component, it is possible to create an event callback function which is chosen among the available ones for the chosen component.

startupFcn

Code that executes automatically after component creation – first function to be called on app launch.

S Y N T A X
`startupFcn(app)`

D E S C R I P T I O N
- Add necessary folders to Matlab path: make all the files and functions inside these folders accessible to Matlab. Assign current folder to working directory by default and set source directory.
- Initialize images: has to be done after `addpath(assets)`.
- Initialize cells: `initCells(app)`.
- Move app to center of screen and delete Loading Panel.

S E E A L S O
`initCells`, `settings`

UIFigureCloseRequest

Code that executes automatically when the main app is closed.

S Y N T A X
`UIFigureCloseRequest(app, event)`

D E S C R I P T I O N
- Closes the associated figure before closing the app

RunButtonPushed

Run button callback.

S Y N T A X
`RunButtonPushed(app, event)`

D E S C R I P T I O N
- Checks if entered parameters are correct.
- Opens a loading bar.
- Launches `assignAndRun` script.
- Plots the model and the meshed model in the SpicePlots tab.
- Enable Save button
- Closes loading bar and moves to SpicePlots tab.

S E E A L S O
`assignAndRun`, `checkMandatoryFields`, `checkFrequencyRange`

MeshdensitySliderValueChanged

Mesh density slider callback.

S Y N T A X
```
MeshdensitySliderValueChanged(app, event)
```

D E S C R I P T I O N
- Changes the Slider value to the closest integer to discretise output range (1-9).

PreviewallButtonPushed

Preview All button callback.

S Y N T A X
```
PreviewallButtonPushed(app, event)
```

D E S C R I P T I O N
- Calls `initFigure`.
- Creates axes in said figure if there are none, otherwise gets the existing axes.
- Plots both the cells and the model in said axes.

S E E A L S O
`initFigure`, `IMSCellPreview`

SaveButtonPushed

Save button callback.

S Y N T A X
```
SaveButtonPushed(app, event)
```

D E S C R I P T I O N
- Calls `saveParameters`.

S E E A L S O
`saveParameters`, `IMSCellPreview`

LoadMatlabFileButton_ParametersPushed

Load Matlab file button callback – from Parameters Tab.

S Y N T A X
```
LoadMatlabFileButton_ParametersPushed(app, event)
```

D E S C R I P T I O N
- Calls `loadParameters`.

S E E A L S O
`loadParameters`

LoadMatlabFileButton_MenuPushed

Load Matlab file button callback – from Menu Tab.

S Y N T A X
`LoadMatlabFileButton_MenuPushed(app, event)`

D E S C R I P T I O N
- Calls `loadParameters`.
- Switches to Parameters Tab if `loadParameters` was successful.

S E E A L S O
`loadParameters`

LoadIMSFolderButtonPushed

Load IMS folder button callback – same for Menu Tab button and Parameters Tab button.

S Y N T A X
`LoadIMSFolderButtonPushed(app, event)`

D E S C R I P T I O N
- Let user choose a directory.
- Opens a loading bar.
- Checks if the folder is named *<name>*`_Statistics.`
- Checks contents of folder.
- If both are correct, adds folder to path and opens the files containing the position and size data of the ellipsoid cells.
- Updates the `cells` variable with the parsed data.
- Switches tabs if necessary.
- Updates displayed information.
- Closes the loading bar.

S E E A L S O
`checkIMSFiles`, `IMSCellPreview`, `cells`

LoadMPHButtonPushed

Load .mph file button callback – same for Menu Tab button and Parameters Tab button.

S Y N T A X
`LoadMPHButtonPushed(app, event)`

D E S C R I P T I O N
- Let user choose a `.mph` file.
- Opens a loading bar.
- Updates the `loadedModel` and `settings` variables with the parsed file data.
- Updates the displayed information.
- Plots a preview of the model in the Parameters Tab.
- Switches Tabs if necessary.
- Closes the loading bar.

S E E A L S O

refreshParametersDisplay, dispFileInfo, countElements, loadedModel, settings, cells

ClearButton_BiocellsPushed

Clear button in Biocells panel callback.

S Y N T A X
```
ClearButton_BiocellsPushed(app, event)
```

D E S C R I P T I O N
- Clears the cells variable.

S E E A L S O
cells, initCells, dispFileInfo

ClearButton_MicroSystemPushed

Clear button in Microsystem panel callback.

S Y N T A X
```
ClearButton_MicroSystemPushed(app, event)
```

D E S C R I P T I O N
- Clears the loadedModel variable.
- Enables adding the loaded cells to the model and saving a copy .mph file.

S E E A L S O
loadedModel, dispFileInfo

BrowseButtonPushed

Browse button callback in Menu Tab.

S Y N T A X
```
BrowseButtonPushed(app, event)
```

D E S C R I P T I O N
- Let user choose the directory where saved files will be written.

S E E A L S O
settings

AddcellstomodelButtonPushed

Add cells button callback in Parameters Tab.

S Y N T A X
```
AddcellstomodelButtonPushed(app, event)
```

D E S C R I P T I O N

- Checks for a loaded cells folder and a loaded .mph file.
- Launches `buildImarisCells` function.
- Refreshes displayed information.

S E E A L S O
`dispFileInfo`, `loadedModel`, `cells`, `buildImarisCells`, checkModelCells

UIAxesButtonDown

Code that executes every time an *axes* object is clicked. Saves the last axes clicked.

S Y N T A X
```
UIAxesButtonDown(app, event)
```

S E E A L S O
`lastAxisClicked`, OpeninexternalfigureMenuSelected

OpeninexternalfigureMenuSelected

Context menu code for *Open in external figure* button when right-clicking on axes.

S Y N T A X
```
OpeninexternalfigureMenuSelected(app, event)
```

D E S C R I P T I O N
- Calls `initFigure`.
- Copies right-clicked axes to external figure.

S E E A L S O
`UIAxesButtonDown`, `initFigure`, `lastAxisClicked`

SaveAllFiguresButtonPushed

Save all figures callback – Spice Plot Tab.

S Y N T A X
```
SaveAllFiguresButtonPushed(app, event)
```

D E S C R I P T I O N
- Opens a loading bar.
- Opens a new minimized figure.
- Copies each *axes* object to said figure and saves each one as a *.fig* file in the set directory.
- Closes the loading bar.

b. Generated

The basic methods that are auto generated and cannot be modified in the App Designer are at the end of the biocad application source code (mlapp script). They are essential for the application to run :

- **`createComponents(app)`** : create all ui components on the mai window (UIFigure) of the interface and initialize their properties, as well as their assigned event callback functions.
- **`app = biocad`** : create the instance of the biocad application interface and returns the biocad Object.
- **`delete(app)`** : delete the UIFigure and in this manner all UI components held by the main figure.

II. Utils functions

This directory holds the external Matlab scripts used in the BIOCAD application.

assignAndRun

Assigns all necessary values to the corresponding fields in the global variable `tlm`, then runs `compute.m`, aswell as `Postprocess.m`.

S Y N T A X
```
assignAndRun(app)
```

S E E A L S O
`settings`, `cells`, `loadedModel`

buildImarisCells

Creates the approximated cells from IMARIS to a COMSOL model.

S Y N T A X
```
[systemModel] = buildImarisCells(app, isSaved)
```

D E S C R I P T I O N
- Verifies if the cells identified in the interface exist and are from an imported Imaris folder.
- Checks if the needed imaris referential point is defined in the COMSOL model.
- Loads a base model : either the current or create a copy if the checkbox *Save to .mph copy* is checked.
- Adds the ellipsoidal cells to the base model with the position shifts to the referential point.
- Creates the gravity center point for each ellipsoidal cell added.
- Runs the created geometries and saves the base model to the file copy if chosen.
- Updates the displayed model's information in the interface (different name if copy).

S E E A L S O
`loadedModel`, `cells`, `AddcellstomodelButtonPushed`

III. SRC directory

This directory holds the backend scripts written by Vincent SENEZ. The backend is the computation heart : it runs the simulation with the parameters entered in the interface and returns result files and axes. For further information, please contact Mr SENEZ.

IV. Assets

The assets are files used in the BIOCAD application, and that are created or taken from a website (free assets). They are stored in the "Assets" folder of the "BIOCAD" folder.

- **`biocad.png`** : Simple logo for the BIOCAD interface.
- **`saveicon.png`** : Icon for the save interface parameters button in the parameters tab.

V. Tricks of the trade

- **Preventing the app from hiding**

All functions that prompt the user to choose a file or a folder (like `uigetfile`, `uigetdir`, and `save`) must be followed by `figure(app.UIFigure)` to bring the application back to the foreground.

- **Avoid changing directories**

Instead of using `cd` to navigate through folders and access function, it is generally easier to use `addpath(<full_path>)` for folders containing functions. To get the full path of a neighboring directory, `genpath` is also a useful tool. For example: `addpath(genpath('./src'))` adds the src folder to the path, making all its contents accessible.

- **Renaming components**

New components created through the interface are automatically given a name that correspond to their type. It is necessary to immediately rename these components to something more verbose that allows for a better understanding of a component's use and location.

- **Quick documentation**

Pressing `F1` with a built-in Matlab function selected opens a window to this function's documentation webpage. Use this to speed up debugging and for quick lookups of unknown functions.

- **Try-catching plots of COMSOL geometry**

When using mphgeom, mphmesh, or another COMSOL plotting function, it is better to enclose it in a try-catch statement such as:

```
try
    mphgeom(app.loadedModel.model, 'geom1', 'facemode', 'off', 'parent', app.UIAxes_MicroSystem);
catch
    warning('COMSOL function "mphgeom" does not quite handle UI axes.');
end
```

This reduces the error printed in the Matlab console down to a single understandable warning message.

- **Autoclose Ngspice**

Using

```
fprintf(fid, '.control\nquit 13\n\n.endc\n.end\n');
```

before the `'.END'` sequence of a file run by NGSPICE makes it automatically close after it has finished running.

Index – Functions

CALLBACK

AboutButtonPushed
BrowseButtonPushed
ClearButton_BiocellsPushed
ClearButton_MicroSystemPushed
LoadIMSFolderButtonPushed
LoadMatlabFileButton_MenuPushed
LoadMatlabFileButton_ParametersPushed
LoadMPHButtonPushed
MeshdensitySliderValueChanged
OpeninexternalfigureMenuSelected
PreviewallButton
RunButtonPushed
SaveAllFiguresButtonPushed
SaveButtonPushed
startupFcn
UIAxesButtonDown
UIFigureCloseRequest

FUNCTIONS

appendChildrenToArray
assignAndRun
checkFrequencyRange
checkIMSFiles
checkMandatoryFields
checkModelCells
countElements
dispFileInfo
getChildren
IMSCellPreview
initCells
initFigure
loadParameters
refreshParametersDisplay
saveParameters
setValuesFromArray